

Term Project: Domain-Specific Intelligent Q&A System

Course: Natural Language Processing with Deep Learning
Weight: 15% of final grade
Group Size: 3–4 students
Presentation Date: Week 15
Presentation Slot: 10 minutes per group (demo) + 5 minutes (Q&A)

Overview

You will design, build, and evaluate a **domain-specific intelligent Q&A system** grounded in the retrieval and evaluation techniques taught in this course. The system must operate on a real-world corpus of your choosing and demonstrate that you understand the pipeline — not just that you can call an API.

Because you are free to use AI tools (Claude Code, Antigravity, ChatGPT, Copilot, or any others), the grading focuses on **understanding and decision-making**, not lines of code. Every technical choice in your system must be something you can explain, defend, and modify on the spot.

Theme & Constraints

Domain (Your Choice)

Pick a real-world domain where a Q&A system would be genuinely useful. Good examples:

- Thai university regulations or student handbooks
- Legal documents (Thai civil code, contracts)
- Medical or clinical guidelines
- Academic papers in a specific subfield
- Company or government policy documents
- Technical product manuals

The corpus must be **real** (not synthetic), contain **at least 50 documents or equivalent**, and be in a language your team can actually read and evaluate.

Mandatory Technical Stack

Every group must implement **all** of the following:

#	Component	Details
1	Text corpus & chunking	Real documents, token-aware chunking with a justified chunk size
2	Dense retrieval	SentenceTransformers embedding model (CPU-compatible)
3	Evaluation harness	At least 2 metrics from the course: BERTScore, BLEU/ROUGE, LLM-as-Judge, RAGAS Faithfulness, RAGAS Answer Relevance

#	Component	Details
4	Ablation study	Compare at least 2 system variants (e.g., dense-only vs. hybrid, with/without reranker) and explain the difference in results

Optional Extensions (improve your grade)

Extension	Recommended if...
BM25 hybrid retrieval + RRF merging	You want stronger retrieval performance
Cross-encoder reranking	You want higher precision on top-k results
LoRA fine-tuning on a small model	Your team has GPU access and wants to explore domain adaptation
GraphRAG or multi-hop retrieval	Your domain has structured relational data
Agentic loop (tool-calling)	Your system needs to handle multi-step queries

You are not required to implement extensions, but a system that only meets the mandatory stack will score in the average range.

Deliverables

Submit all three items before your Week 15 presentation slot:

1. Code Repository

A clean, runnable codebase with:

- A **README.md** explaining how to install dependencies and run the system
- A working demo script or notebook (**demo.ipynb** or **demo.py**)
- Your evaluation script with results saved to a file

2. Presentation Slides (10 slides max)

Structure your slides as follows:

Slide	Content
1	Title, team members, domain chosen
2	Problem motivation — why this domain needs a Q&A system
3	System architecture diagram (your full pipeline)
4–5	Key technical decisions (chunking strategy, embedding model, retrieval approach)
6–7	Ablation results — your table or chart comparing variants
8	Evaluation results with at least 2 metrics

Slide	Content
9	Failure analysis — what breaks and why
10	AI Audit highlights (3 most important decisions + 1 error catch)

No code dumps on slides. Architecture diagrams and result tables only.

3. AI Audit (AI_AUDIT.md)

This is a **separate deliverable** submitted alongside your code. See the AI Audit section below for the exact format. The audit is graded independently using an automated checker — completeness and technical honesty matter more than polish.

AI Policy

You **may** use any AI tool for any part of this project. This is intentional. The constraint is that you must be able to:

- 1. Explain every component of your system to a technical audience
- 2. Trace every design decision (chunk size, embedding model, metric choice) to a reason
- 3. Answer an individual technical question during the presentation without AI assistance

Using AI to generate boilerplate code is fine. Using AI to make decisions you cannot explain is a grading risk — the Q&A will expose it.

AI Audit Instructions

Create a file named AI_AUDIT.md in your repository root. Fill it out honestly using the structure below. Vague or generic answers score zero — the checker looks for specific technical language tied to your actual system.

```
# AI Audit — [Your Project Title]
**Team:** [Names]
**AI Tools Used:** [List all tools]

---

## 1. Tool Inventory

| AI Tool | Primary Use in This Project |
|---|---|
| e.g., Claude Code | Generated BM25 retrieval class, debugged RRF merge function |
| e.g., ChatGPT | Explained cross-encoder vs. bi-encoder tradeoff |

---
```

2. Prompt Log (minimum 5 entries)

For each significant AI interaction, record the actual prompt and what you did with the output.

Entry 1

****Prompt sent:****

> [paste the exact prompt you gave the AI]

****What AI generated:****

[Summarize or paste the key output]

****What you changed or rejected:****

[Be specific – did you change variable names, fix an incorrect API call, restructure the logic, reject a suggested approach?]

[Repeat for entries 2–5+]

3. Decision Journal (minimum 5 decisions)

For each key design decision, record who made it and why.

Use: [Human] / [AI-suggested, Human-approved] / [AI-decided]

#	Decision	Owner	Reason (in your own words)
1	Chunk size = 512 tokens	Human	Longer chunks lost precision in our legal domain; tested 256 vs 512 vs 1024
2	...	...	...

4. Error Catch Log (minimum 2 entries)

Describe cases where the AI was technically wrong.

Error 1

****What the AI said:****

[Quote or summarize the incorrect suggestion]

****Why it was wrong:****

[Cite the course concept or technical reason – e.g., "AI used dot product without normalization, which is incorrect for cosine similarity retrieval"]

****How you fixed it:****

[What you actually did]

[Repeat for error 2+]

5. Contribution Map

Team Member	Human Contribution	AI Tools Used
[Name 1]	Corpus collection, chunking strategy	Claude Code
[Name 2]	BM25 + RRF implementation	GitHub Copilot
[Name 3]	Evaluation harness, BERTScore analysis	ChatGPT
[Name 4]	Presentation, ablation design	Antigravity

6. What Would Break? (Answer WITHOUT using AI)

Answer all three questions in your own words.

****a)**** If you removed the reranker (or your second retrieval stage) from your pipeline, what specifically changes in output quality and why?

****b)**** Why did you chunk your documents at [X] tokens? What would happen to retrieval quality if you used 2× or $\frac{1}{2}$ that size?

****c)**** Your ablation compares two system variants. Pick the result that surprised you most and explain the mechanism behind it — not just "variant A scored higher" but why technically.

Standardized AI Audit Generation Prompt

Everyone must use this exact prompt to generate their **AI_AUDIT.md** skeleton. Copy it into Claude Code, Antigravity, or any AI tool, fill in the **[BRACKETS]** with your project details, then run it. The AI will produce a pre-structured skeleton — you then go through every **[FILL IN: ...]** placeholder and replace it with your real content.

Using this shared prompt ensures all audits have the same format and can be checked consistently.

You are helping me create an AI Audit document for my NLP course term project.

My project details:

- Project title: [TITLE]
- Team members: [NAME 1], [NAME 2], [NAME 3], [NAME 4]
- Domain: [e.g., Thai university regulations / legal documents / medical guidelines]
- AI tools we used: [list all tools and their primary role]
- Corpus: [describe your documents – what they are, how many, what language]
- Embedding model chosen: [e.g., sentence-transformers/paraphrase-

multilingual-mpnet-base-v2]

- Chunk size and overlap: [e.g., 512 tokens with 50-token overlap]
- Retrieval approach: [e.g., dense-only / BM25 + dense hybrid with RRF / + cross-encoder reranker]
- System variants compared in ablation: [e.g., Variant A: dense-only vs. Variant B: dense + BM25 hybrid]
- Evaluation metrics used: [e.g., BERTScore + LLM-as-Judge]
- Optional extensions implemented: [e.g., none / LoRA fine-tuning / GraphRAG]

Generate a complete AI_AUDIT.md file in GitHub-flavored Markdown with these exact six sections:

1. Tool Inventory – a table listing each AI tool and its primary role in this project
2. Prompt Log – 5 entries, each with subsections: "Prompt sent", "What AI generated", "What you changed or rejected". Pre-fill the project context in the headers but leave the actual prompts and outputs as [FILL IN: paste the real prompt you sent] and [FILL IN: summarize what the AI produced]
3. Decision Journal – a table of 5 rows for key design decisions with columns: #, Decision, Owner ([Human] / [AI-suggested, Human-approved] / [AI-decided]), Reason. Pre-fill the Decision column with the actual decisions from my project details above (chunk size, embedding model, retrieval approach, evaluation metrics, ablation design). Leave Owner and Reason as [FILL IN: ...]
4. Error Catch Log – 2 entries with subsections: "What the AI said", "Why it was wrong", "How you fixed it". Leave all content as [FILL IN: ...] – do NOT invent errors
5. Contribution Map – a table with columns: Team Member, Human Contribution, AI Tools Used. Pre-fill the team member names. Leave contributions as [FILL IN: ...]
6. What Would Break? – three questions pre-filled with our specific system details:
 - a) Refer to our actual second retrieval stage or reranker if we have one, or dense retrieval if not
 - b) Reference our actual chunk size of [X] tokens
 - c) Reference our actual two ablation variants by name

IMPORTANT RULES:

- Do NOT fabricate or invent any prompt content, error descriptions, or decisions
- Every piece of real content must be a [FILL IN: ...] placeholder
- Use our actual project details (model name, chunk size, domain, etc.) wherever the structure allows
- Output ONLY the markdown file – no explanations before or after

After running this prompt, you will have a skeleton file. Replace every [FILL IN: ...] with real content from your project. The final submitted AI_AUDIT.md must have **no remaining placeholders** — any unfilled section scores zero.

Rubric

Criterion	Points	What Earns Full Credit
System works on real corpus	15	Live demo runs, corpus is non-trivial, chunking is justified
Technical depth	25	Pipeline goes beyond single API call; team can explain each component
Ablation quality	20	Results table is correct; interpretation explains the mechanism, not just the number
Evaluation rigor	15	≥ 2 metrics used correctly; team understands what each metric measures
AI Audit score	15	Audit is specific, honest, and shows real technical engagement
Presentation & individual Q&A	10	Slides are clear; each member answers their individual question correctly

Total: 100 points → 15% of course grade

The individual Q&A accounts for a portion of the Presentation score. Each team member will receive one technical question during the presentation. The question is assigned randomly and must be answered individually without group assistance.

Presentation Format

Total slot: 15 minutes per group (10 min presentation + 5 min Q&A)

Part 1 — Presentation (10 minutes)

Time	Activity
0:00 – 1:00	Problem intro & domain motivation
1:00 – 4:00	System walkthrough (architecture + key decisions)
4:00 – 7:00	Live demo
7:00 – 10:00	Ablation results, evaluation scores, failure analysis

Part 2 — Individual Q&A (5 minutes)

After the presentation ends, each team member receives **one** technical question assigned randomly. You answer individually — no group discussion, no AI assistance. Each member has approximately **1 minute** to answer.

Questions are drawn from a bank covering: retrieval mechanics, evaluation metrics, system design decisions, and core NLP concepts applied to your specific system. The question will be about your system — not generic trivia. If you built it, you should be able to answer it.

Keep the demo short and focused. A system that works on 3 example queries is better than a broken demo of 10 queries.

Advice

On choosing your domain: Pick something at least one team member genuinely cares about. You will spend many hours with this corpus. A motivated team that chose an interesting domain consistently outperforms a team that picked something "easy."

On the ablation: Do not run it the night before. The ablation is how you demonstrate that you understand your system. Run early, interpret the results, and prepare to explain them.

On the AI Audit: Write it as you build, not at the end. Reconstructing prompts from memory produces vague logs that score poorly.

On the Q&A: The question will be about your system specifically — not a generic NLP trivia question. If you built it, you should be able to answer it. If your teammate built it and you cannot explain it, that is a grading signal.